



AI VIET NAM – AI Conquer 2026

Trợ lý Hỏi–Đáp Tài liệu khóa học Project RAG Chatbot

Báo cáo kỹ thuật theo hướng Đi làm (Application) — Project Chatbot AI

Nhóm: **Overfitting Club**

I. Tổng quan báo cáo

Sản phẩm là gì. Nhóm xây một **trợ lý hỏi–đáp tài liệu** cho khóa học: học viên hỏi bằng lời thường (Việt hoặc Anh), hệ thống tìm trong tài liệu rồi trả lời ngay trên màn hình, chữ hiện dần từng phần. Mỗi câu trả lời lấy từ chính tài liệu và *trích rõ nguồn* (tài liệu nào, trang nào) để người đọc kiểm chứng và hạn chế trả lời sai. Báo cáo tập trung phần lõi: tìm đúng đoạn và viết câu trả lời bám nguồn.

Vì sao cần đến nó. Tài liệu một khóa học thường dày hàng trăm trang. Tìm bằng từ khóa chỉ khớp đúng chữ, không hiểu ý câu hỏi và không cho ra câu trả lời gọn. Nhóm cần công cụ vừa hiểu ý hỏi để tìm đúng đoạn, vừa viết lại thành câu trả lời bám tài liệu.

Cách tiếp cận. Đây là bài toán **RAG** (Retrieval-Augmented Generation): *tìm* các đoạn liên quan trước, rồi mới *nhờ mô hình ngôn ngữ viết* câu trả lời dựa trên đúng các đoạn đó, nên câu trả lời gần nguồn thật thay vì tự bịa.

Báo cáo trình bày gì. Toàn bộ trợ lý được gói trong một dịch vụ web (Django) lo trọn hai việc:

- **Nạp tài liệu** (chạy ngầm): đọc chữ trong file, kể cả trang scan phải nhận dạng ảnh → cắt thành đoạn nhỏ → chuyển mỗi đoạn thành vector số (embedding) để tìm theo ý nghĩa → cất vào kho tìm kiếm; việc nền xếp hàng bằng Celery. Ngoài ra còn dựng một *đồ thị tri thức* cho câu hỏi suy luận: đã hiện thực và bật sẵn, nhưng an toàn kiểu fail-safe và chỉ chạy khi có hạ tầng PostgreSQL/Neo4j (công cụ: LightRAG).
- **Trả lời** (tức thời): với mỗi câu hỏi, tìm các đoạn liên quan theo hai đường hỗ trợ. Đường chính tìm song song bằng khớp từ khóa và khớp ý nghĩa, hợp nhất rồi lọc bớt đoạn kém liên quan trên kho tìm kiếm; bước xếp lại thứ tự tinh hơn đã hiện thực nhưng chưa bật. Đường bổ sung tra thực thể và quan hệ trên đồ thị tri thức cho câu hỏi suy luận nhiều bước (công cụ: LightRAG [1]). Có các đoạn rồi, mô hình ngôn ngữ tổng hợp thành câu trả lời và gửi về theo kiểu hiện dần từng phần (công cụ: mô hình Gemini, điều phối qua Google ADK, truyền qua SSE).

Ở mỗi phần sau, nhóm trình bày *cách xử lý* trước, rồi mới nêu *công cụ*. Trợ lý tổ chức thành nhiều lớp rõ ràng: từ tiếp nhận yêu cầu qua web, điều phối, tới lõi tìm kiếm và sinh câu trả lời. Các thành phần hạ tầng đi kèm — kho tìm kiếm (OpenSearch), nơi lưu file (S3), mô hình ngôn ngữ (Gemini), hàng đợi việc nền (Redis), cơ sở dữ liệu bản ghi tài liệu (MariaDB) — chỉ nhắc đủ hiểu, không phải trọng tâm.

Một câu định vị sản phẩm

Trợ lý hỏi–đáp tài liệu theo kiến trúc RAG: nạp tài liệu khóa học thành kho tìm kiếm được, rồi mỗi câu hỏi sẽ tìm các đoạn liên quan trong kho (bổ sung từ đồ thị tri thức LightRAG khi cần quan hệ và hạ tầng đồ thị sẵn sàng), viết câu trả lời bám nguồn và trả về theo kiểu hiện dần từng phần.

Mục lục

I.	Tổng quan báo cáo	1
II.	Định nghĩa bài toán	4
III.	Dữ liệu sử dụng	4
IV.	Tiền xử lý và chuẩn bị dữ liệu	5
V.	Phương pháp và mô hình sử dụng	6
V.1.	Truy xuất hybrid (BM25 + kNN) và RRF	6
V.2.	Truy xuất đồ thị tri thức (LightRAG, đã hiện thực, cấu hình bật)	8
V.3.	Sinh câu trả lời bằng Google ADK	8
VI.	Luồng huấn luyện và suy luận	9
VI.1.	Luồng “huấn luyện” = Ingest (xử lý nền)	9
VI.2.	Luồng suy luận = Chat (thời gian thực, SSE)	9
VII.	Công cụ và công nghệ sử dụng	10
VIII.	Thực nghiệm và đánh giá mô hình	11
VIII.1.	Đánh giá LightRAG/KG riêng (A/B trên câu hỏi quan hệ)	13
IX.	Phân tích lỗi và hạn chế	14
X.	Kết luận và hướng phát triển	15
XI.	Tài liệu tham khảo	17
	Phụ lục	18

II. Định nghĩa bài toán

Loại bài toán. *Retrieval-Augmented Question Answering* (RAG) trên kho tài liệu khóa học.

Đầu vào. Một câu hỏi viết bằng lời thường, chủ yếu tiếng Việt hoặc tiếng Anh. Người dùng có thể (tùy chọn) gắn mã hội thoại để giữ ngữ cảnh các lượt trước và đính kèm tài liệu riêng cho lượt hỏi.

Đầu ra. Câu trả lời bằng văn bản, dựng từ tài liệu đã nạp và hiện dần từng phần (streaming qua SSE), kèm **trích nguồn** (tên tài liệu, số trang, điểm liên quan) để người đọc kiểm chứng.

Hai luồng con. Một là *nạp tài liệu* (ingest): biến tài liệu thô thành kho tri thức tìm kiếm được. Hai là *trả lời* (answering): tìm các đoạn liên quan rồi sinh câu trả lời. Câu hỏi cần suy luận theo quan hệ được bổ sung thực thể và quan hệ từ đồ thị tri thức, chỉ khi hạ tầng đồ thị sẵn sàng (công cụ: OpenSearch cho tìm kiếm, đồ thị tri thức LightRAG cho quan hệ).

Tiêu chí thành công.

- Tìm được đúng đoạn tài liệu liên quan đến câu hỏi.
- Câu trả lời *bám sát nguồn*, không tự bịa (groundedness/faithfulness).
- Độ trễ chấp nhận được, quan trọng nhất là thời gian tới chữ đầu tiên (time-to-first-token) vì giao diện hiện dần từng phần.
- Chạy trơn vẹn demo từ nạp tài liệu đến nhận câu trả lời.

Định nghĩa bài toán

Retrieval-augmented question answering: đầu vào là câu hỏi ngôn ngữ tự nhiên (đa ngôn ngữ); module truy xuất các đoạn liên quan từ kho tri thức (OpenSearch, và đồ thị tri thức LightRAG khi cần quan hệ và hạ tầng KG sẵn sàng) rồi sinh câu trả lời bám sát nguồn, trả về qua SSE streaming.

III. Dữ liệu sử dụng

Nguồn. Tài liệu khóa học ở hai định dạng: **PDF** (kể cả PDF scan hoặc ảnh, phải nhận dạng chữ bằng OCR) và **Word (DOCX)**. Module lấy danh sách tài liệu cần xử lý từ hai bảng dùng chung, chỉ ánh xạ để đọc chứ không sở hữu hay đổi cấu trúc.

Lưu trữ.

- **File gốc:** lưu trên **S3** (tương thích MinIO); module tải file về qua thư viện chuẩn của AWS.
- **Kho tri thức** (OpenSearch là chính; đồ thị tri thức là thành phần bổ trợ, chỉ vận hành khi có hạ tầng PostgreSQL/Neo4j):

- **OpenSearch 2.19.1** (chính) — chỉ mục rag-index (vector kNN + text BM25), phụ trợ document-summary-index và chatbot-chat-history (trí nhớ dài hạn).
- **Đồ thị tri thức LightRAG** [1] (bổ trợ) — lưu *thực thể và quan hệ* cho câu hỏi cần lần nhiều bước. Đã hiện thực và bật sẵn theo cấu hình; nếu lỗi thì nuốt và ghi log, không làm tài liệu hỏng. Chỉ thật sự chạy và đo được khi có hạ tầng PostgreSQL/Neo4j [2] cùng phần mở rộng vector cho PostgreSQL [3].

Quy mô (bộ dữ liệu thực nghiệm hiện tại). Bộ thử nghiệm gồm **9 tài liệu** khóa học AIO 2026: Python cơ bản (vòng lặp; rẽ nhánh và chatbot theo luật), SQL và ERD, Coding Methodology, Image & Video Data, Teamwork Tools (Agile/Scrum/Jira), hướng dẫn làm video, và chính sách bảo mật. Mỗi tài liệu dài khoảng 33–101 trang, được cắt thành tổng cộng **782 chunk** con rồi nạp vào kho tìm kiếm: mỗi tài liệu giữ một bản ghi cha kèm nhiều bản ghi con theo từng đoạn nhỏ.

Vấn đề dữ liệu. Trang scan chất lượng thấp, bảng và công thức khó nhận dạng chữ; độ dài tài liệu không đều, tiếng Việt có dấu, một số nội dung trùng lặp. Nhóm xử lý một phần ở khâu tiền xử lý (mục IV.) và phân tích thêm ở mục IX..

IV. Tiền xử lý và chuẩn bị dữ liệu

Đây là khâu quyết định chất lượng RAG. Nhóm dựng pipeline chạy nền: người dùng tải tài liệu lên là hệ thống tự xử lý, không bắt chờ. Luồng được điều phối tại `app/pipelines/ingest.py` (hàm `run_ingest_pipeline`) và chạy nền qua `app/tasks/ingest.py` (hàng đợi Celery, broker Redis). Pipeline gồm 6 bước.

1) Lấy file gốc. Nhóm tải đúng file người dùng đã lên từ kho lưu trữ đám mây, theo tên ghi trong bản ghi tài liệu. Việc kết nối do lớp tiện ích S3 dùng chung đảm nhiệm (tự đọc cấu hình từ môi trường).

2) Trích chữ theo từng trang. Nhóm đọc từng trang: trang có sẵn chữ số hóa thì đọc thẳng; trang gần như trống chữ (dưới 20 ký tự) hoặc chứa ảnh lớn (từ 5000 byte) thường là scan nên nhận diện chữ trên ảnh, nếu không ra chữ thì giữ phần đọc thẳng để không mất dữ liệu. File Word đổi sang PDF trước cho đồng nhất. Công cụ: đọc chữ bằng `pypdf`, nhận diện ảnh (OCR) bằng Gemini, đổi định dạng bằng LibreOffice (`soffice`, giới hạn 120s).

3) Cắt tài liệu thành đoạn nhỏ. Tài liệu dài được cắt thành đoạn ngắn, mỗi đoạn khoảng 800 ký tự và gói nhau 120 ký tự để không đứt ý. Mỗi đoạn còn gắn một dòng ngữ cảnh ở đầu (thuộc tài liệu nào, phần nào) để khi tách ra vẫn tự hiểu, nhờ đó tìm lại đúng hơn. Việc cắt đoạn dùng bộ tách văn bản của LangChain [4].

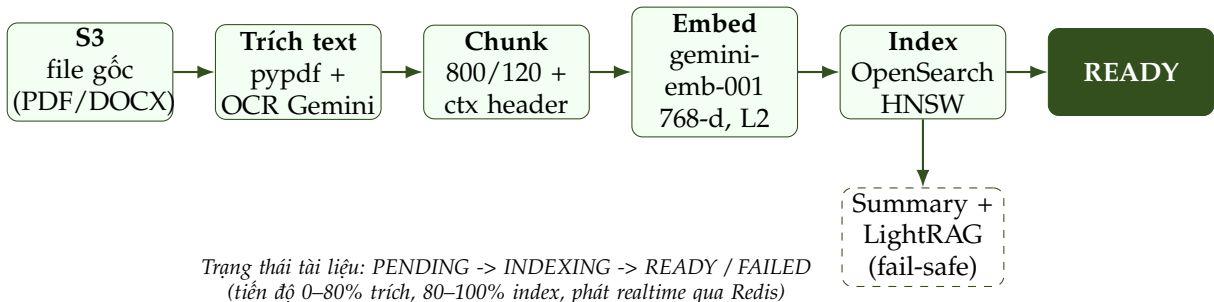
4) Chuyển đoạn văn thành vector. Để máy so khớp theo ngữ nghĩa, mỗi đoạn được biểu diễn thành dãy số (vector) **768 chiều**, chuẩn hóa độ dài về cùng thang (chuẩn hóa L2) để so sánh công bằng và đánh dấu là dữ liệu truy hồi. Các đoạn xử lý theo lô 50 cho nhanh. Việc tạo vector dùng mô hình nhúng `gemini-embedding-001`.

5) Lưu vào kho tìm kiếm theo cấu trúc cha-con. Mỗi tài liệu lưu thành một bản ghi cha (thông tin chung) kèm nhiều bản ghi con (nội dung đoạn, vector, số trang), liên kết cha-con để vừa tìm ở mức đoạn vừa truy ngược ra tài liệu gốc. Khi nạp lại, nhóm ghi bản mới xong mới xóa

bản cũ nên tìm kiếm không hụt dữ liệu giữa chừng. Kho lưu và tìm kiếm là OpenSearch.

6) Hai việc phụ, hỏng cũng không sao. Sau phần chính, nhóm làm thêm hai việc bổ trợ, cố ý tách riêng để nếu lỗi thì tài liệu vẫn dùng được. Một là lưu bản tóm tắt ngắn (tối đa 200 ký tự) để tra nhanh ở mức tài liệu. Hai là dựng đồ thị tri thức: đọc toàn văn, rút thực thể và quan hệ rồi lưu thành mạng lưới liên kết, giúp trả lời tốt hơn cho câu hỏi mang tính quan hệ. Việc này chỉ chạy khi có sẵn hạ tầng cơ sở dữ liệu, luôn sau bước lưu kho tìm kiếm chính, nạp lại cùng tài liệu thì làm mới sạch chữ không nhân đôi; lỗi cũng không chặn trạng thái sẵn sàng. Công cụ: rút thực thể–quan hệ bằng mô hình gemini-2.5-flash, lưu đồ thị bằng LightRAG [1] trên nền PostgreSQL/Neo4j.

Máy trạng thái tài liệu (app/enums/document_status.py): PENDING → INDEXING → READY hoặc FAILED (lỗi ngoại vi S3/Gemini/OpenSearch). Tiến độ indexed_percent (0–100) phát realtime qua Redis (khâu trích 0–80%, khâu index 80–100%). Sơ đồ ở Hình 1.



Hình 1: Pipeline Ingest: lấy file gốc → trích chữ (pypdf, OCR Gemini) → cắt đoạn kèm ngữ cảnh → tạo vector 768 chiều → lưu cha-con vào OpenSearch. Hai việc phụ (tóm tắt, đồ thị LightRAG) chạy tách riêng, lỗi cũng không chặn trạng thái READY.

V. Phương pháp và mô hình sử dụng

Phần “AI” cốt lõi làm hai việc. Trước hết, hệ thống *tìm* các đoạn tài liệu liên quan bằng hai đường bổ sung nhau: đường chính luôn bật (khớp từ khóa và khớp ý nghĩa), và đường phụ dựa trên đồ thị tri thức cho câu hỏi cần quan hệ. Sau đó, một agent *tổng hợp* các đoạn thành câu trả lời.

Ba thành phần: (1) **truy xuất hybrid** BM25 + kNN trên OpenSearch (app/rag) — đường chính, luôn bật; (2) **truy xuất đồ thị tri thức** bằng LightRAG [1] (app/lightrag) — đường phụ, đã hiện thực và bật cấu hình, chỉ chạy khi có hạ tầng đồ thị; (3) **sinh câu trả lời** bằng agent qua Google ADK (app/adk) — gộp ngữ cảnh đường chính, bổ sung từ đường đồ thị khi câu hỏi cần quan hệ.

V.1. Truy xuất hybrid (BM25 + kNN) và RRF

Điểm xuất phát. Cách đơn giản nhất là tìm theo *từ khóa*: chỉ khớp đúng chữ trong đoạn (công cụ: BM25); nhóm lấy đây làm mốc so sánh.

Truy xuất hybrid. Để tìm đúng hơn, nhóm chạy song song hai cách trên cùng kho: khớp *từ*

khóa và *khớp ý nghĩa* (câu hỏi chuyển thành vector rồi so khớp). Mỗi cách cho một bảng xếp hạng; hai bảng được trộn thành danh sách chung, ưu tiên đoạn được cả hai xếp cao. Cách trộn dùng **Reciprocal Rank Fusion (RRF)**, cộng điểm theo thứ hạng ($k = 60$). Công cụ: từ khóa dùng **BM25**, ý nghĩa dùng **kNN vector** trên chỉ mục **HNSW** của OpenSearch [5].

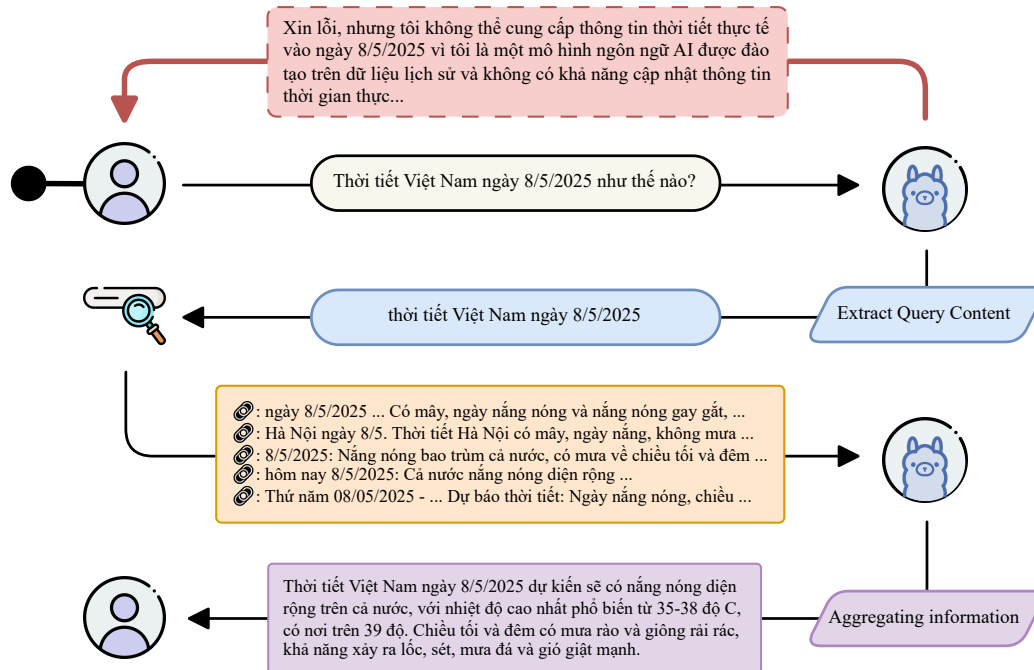
Chọn lọc lại (rerank). Từ danh sách trộn, nhóm lấy 30 đoạn dẫn đầu rồi chấm lại theo mức liên quan, giữ 5 đoạn tốt nhất đưa cho mô hình trả lời. Bước này dùng mô hình so khớp câu hỏi–đoạn (cross-encoder bge-reranker-v2-m3), gọi qua dịch vụ cấm ngoài (timeout 15s), chỉ giữ đoạn vượt ngưỡng 0.3.

Bước này *tùy chọn* và có dự phòng: nếu chưa cấu hình dịch vụ chấm lại, hệ thống giữ nguyên thứ tự đã trộn, có thể lọc thêm bằng ngưỡng tương đồng tối thiểu như tín hiệu chống trả lời bịa (mặc định 0.0, cấu hình mẫu gợi ý 0.80).

Lưu ý về phạm vi

Tính năng **viết lại câu hỏi** (query rewriting) mới được *khai báo* trong cấu hình, chưa hiện thực trong code, nên không tham gia luồng tìm kiếm; nhóm xếp vào *hướng phát triển* (mục X.).

Tóm lại, một câu hỏi đi qua: chuyển thành vector, chạy song song khớp từ khóa và ý nghĩa, trộn bằng RRF ($k = 60$), chấm lọc lại (tùy chọn) rồi trả về 5 đoạn tốt nhất (Hình 2). Đó là đường chính; đường thứ hai dựa trên đồ thị tri thức, trình bày ở mục V.2..



Hình 2: Pipeline truy xuất: khớp từ khóa (BM25) và khớp ý nghĩa (kNN) được trộn bằng RRF rồi lọc theo ngưỡng tương đồng; bước chấm lọc lại (rerank) là tùy chọn.

V.2. Truy xuất đồ thị tri thức (LightRAG, đã hiện thực, cấu hình bật)

Ngoài đường chính, module dựng thêm đường *đồ thị tri thức*: thay vì tìm các đoạn rời rạc, hệ thống lập mạng lưới *thực thể* và *quan hệ* rút từ tài liệu để truy theo mối liên hệ. Đường này hỗ trợ cho đường chính (công cụ: **LightRAG** [1], theo tinh thần **GraphRAG** [6]).

Đường đồ thị đã bật trong cấu hình và có cơ chế an toàn: mọi lỗi được ghi log và bỏ qua, không làm tài liệu rơi vào trạng thái thất bại. Nó chỉ truy hồi khi có hạ tầng đồ thị (PostgreSQL và Neo4j) và khi agent gọi tới.

Khi nạp tài liệu. Toàn văn được đưa vào để dựng đồ thị: một mô hình ngôn ngữ đọc văn bản và *rút ra thực thể cùng quan hệ* (mô hình: gemini-2.5-flash). Kết quả lưu hai nơi: đồ thị trong Neo4j, dữ liệu kèm theo trong PostgreSQL. Nạp lại cùng tài liệu sẽ xóa bản cũ rồi chèn mới, không bị trùng.

Khi trả lời. Hệ thống truy vấn kiểu *trộn*: gộp kết quả tìm theo đoạn văn và kết quả đi theo thực thể–quan hệ trên đồ thị. Đường đồ thị chỉ trả về *ngữ cảnh thô* (thực thể, quan hệ, đoạn liên quan) chứ không tự viết câu trả lời; việc đó để cho agent (mục V.3.). Do kết quả đồ thị không kèm nguồn của đoạn nên phần này *không tạo trích dẫn*.

Vì sao cần đồ thị. Tìm theo đoạn văn rất mạnh khi tra một sự kiện đơn lẻ, nhưng yếu với câu hỏi *quan hệ* (“X ảnh hưởng Y thế nào”, “cái gì liên kết A với B”) và suy luận nhiều bước qua nhiều tài liệu. Đồ thị thực thể–quan hệ bù đúng điểm yếu đó; hạ tầng lưu trữ chỉ bật khi cần.

V.3. Sinh câu trả lời bằng Google ADK

Có các đoạn liên quan rồi, một *agent* đọc chúng và viết thành câu trả lời, tự quyết gọi công cụ tìm kiếm nào cho từng câu hỏi. Phần này dùng mô hình gemini-2.5-flash, điều phối qua bộ khung **Google ADK** [7]:

- **Agent** có bốn công cụ: tìm đoạn văn theo đường chính (`search_knowledge_base`); tìm thực thể–quan hệ trên đồ thị (`search_knowledge_graph`); lấy đường dẫn tài liệu (`get_document_url`); vẽ sơ đồ tư duy (`create_mind_map`). Hướng dẫn phân vai rõ: tìm đoạn văn cho câu hỏi tra cứu sự kiện, tìm trên đồ thị cho câu hỏi quan hệ hoặc nhiều bước. Hai công cụ tìm kiếm luôn đăng ký sẵn dù đường đồ thị bật hay tắt; bật/tắt xử lý lúc chạy, không gỡ công cụ.
- **Bộ chạy (runner)** phát câu trả lời theo kiểu hiện dần từng phần và tách riêng từng phiên trò chuyện theo mã phiên.
- **Bộ lập kế hoạch (planner)** cho mô hình *suy nghĩ* thành các bước trước khi trả lời (tính năng của Gemini 2.5 trở lên), kèm một lớp xử lý dòng chữ để tránh lặp khi trả về hiện dần.

Trí nhớ và ngữ cảnh. Để bám mạch trò chuyện, mỗi lượt hỏi được ghép ba nguồn: **lịch sử gần đây** (10 lượt gần nhất); **trí nhớ dài hạn** (3 đoạn hội thoại cũ liên quan nhất, so khớp ý nghĩa với ngưỡng 0.5); và **ngữ cảnh tài liệu** (5 đoạn liên quan nhất từ đường chính).

Lý do chọn các hướng này. Nhóm dùng mô hình đã huấn luyện sẵn nên *không phải tự huấn luyện*. Kết hợp từ khóa và ý nghĩa, trộn bằng RRF rồi chấm lọc lại, cân bằng giữa tìm được nhiều đoạn đúng và trùng đoạn thật sự liên quan. Bộ khung ADK chuẩn hóa việc quản lý agent, công cụ và luồng trả từng phần. Phần chuyển câu hỏi và đoạn văn thành vector hợp với truy xuất theo ý nghĩa và đa ngôn ngữ.

Đường đồ thị (**LightRAG/GraphRAG** [1], [6]) được thêm làm *bổ trợ* vì tìm theo đoạn văn thuần còn yếu ở câu hỏi quan hệ và nhiều bước. Đổi lại, nhóm phải dựng thêm hạ tầng đồ thị và chịu chi phí gọi mô hình trích thực thể lúc nạp; nhóm chấp nhận để có khả năng suy luận qua nhiều tài liệu. Module cũng có thể chuyển sang chạy mô hình cục bộ bằng **Ollama** nếu cần.

VI. Luồng huấn luyện và suy luận

RAG **không huấn luyện model mới**, nên nhóm hiểu lại hai luồng theo hệ thống. “Huấn luyện” là **Ingest**: xây kho tri thức, chạy ngầm bất đồng bộ. “Suy luận” là **Chat**: trả lời người dùng thời gian thực qua SSE.

VI.1. Luồng “huấn luyện” = Ingest (xử lý nền)

Có tài liệu mới, module nhận qua route nội bộ, trả 202 ngay và đẩy việc nặng sang tác vụ nền (Celery). Tác vụ chạy tuần tự: trích text hoặc OCR → chia đoạn, tạo vector → đưa vào OpenSearch. Xong, tài liệu chuyển READY.

Sau đó là bước phụ an toàn: dựng tóm tắt và đồ thị tri thức bằng **LightRAG** [1] theo lối xóa rồi chèn lại, chỉ chạy khi có hạ tầng PostgreSQL/Neo4j. Lỗi ở đây đều bỏ qua, *không bao giờ* khiến tài liệu chuyển FAILED. Chi tiết ở mục IV. và Hình 1.

VI.2. Luồng suy luận = Chat (thời gian thực, SSE)

Nhóm điều phối luồng Chat trong một dịch vụ chat riêng, gồm các bước:

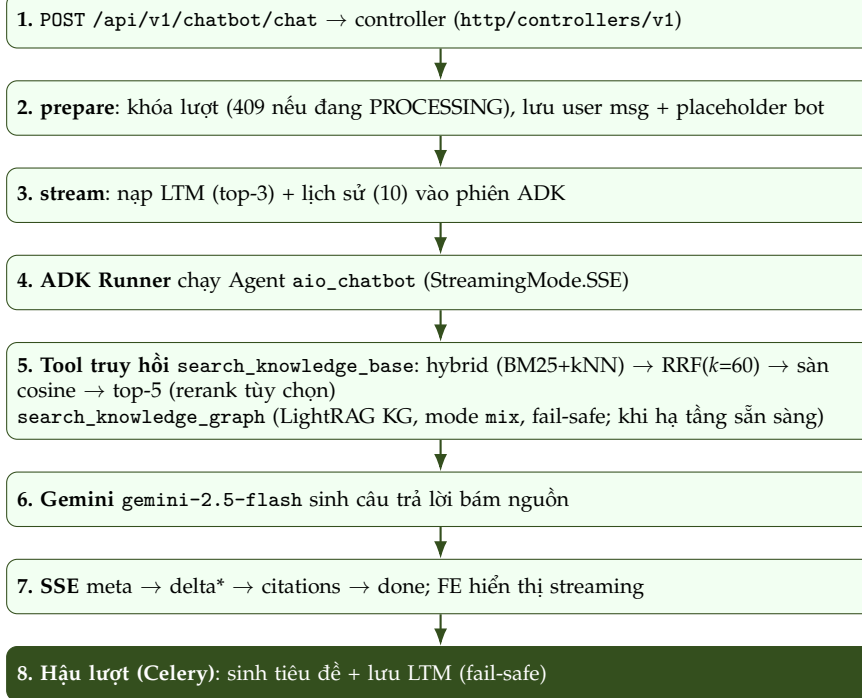
1. Giao diện gửi câu hỏi tới điểm cuối chat công khai (POST /api/v1/chatbot/chat).
2. **Chuẩn bị** (đồng bộ): mở/lấy hội thoại rồi khóa lượt tránh chạy trùng. Nếu lượt trước còn xử lý, trả **409 Conflict**; nếu không, lưu tin người dùng và tạo sẵn tin trả lời ở trạng thái PROCESSING.
3. **Truyền dòng**: nạp bộ nhớ dài hạn và lịch sử vào phiên, rồi để agent (Google ADK) xử lý. Agent tra cứu qua truy hồi hybrid (BM25 kết hợp vector, RRF, sản cosine; tùy chọn rerank); câu cần suy luận nhiều bước thì gọi thêm đồ thị tri thức LightRAG khi hạ tầng sẵn sàng. Cuối cùng Gemini sinh câu trả lời, trả từng mẫu.
4. **Giao thức SSE**: các mẫu được gửi theo thứ tự meta → (delta | thinking | citations)* → mindmap_pending? → mindmap? → done | error. Mỗi mẫu là một dòng JSON, giữ nguyên tiếng Việt và tắt đệm để chữ hiện dần trên màn hình.

5. **Sau lượt** (chạy nền, an toàn): sinh tiêu đề hội thoại bằng gemini-2.5-flash và lưu lượt vào bộ nhớ dài hạn dạng vector 768-d. Trạng thái tin chuyển PROCESSING → SUCCESS hoặc ERROR.

Giao thức SSE thực tế (text/event-stream)

```
data: {"type":"meta","conversation_id":12,"message_id":85,"citations":[...] }
data: {"type":"thinking","content":"..."}           # khi Gemini 2.5 bat thinking
data: {"type":"delta","content":"doan van ban tra loi ..."}
data: {"type":"delta","content":"... tiếp theo ..."}
data: {"type":"citations","citations":[{"original_name":"...","page":7,"score":0.82}]}
data: {"type":"mindmap_pending"}                   # neu user yeu cau so do tu duy
data: {"type":"mindmap","mind_map":{"title":"...","nodes":[...]}}
data: {"type":"done","status":"success","message_id":85,"total_tokens":1234}
```

Hình 3 tóm tắt toàn bộ luồng suy luận.



Hình 3: Luồng Chat (SSE): nhận yêu cầu, chuẩn bị lượt, nạp ngữ cảnh, agent gọi truy hồi hybrid, Gemini sinh và trả dần câu trả lời, rồi bước sau lượt.

VII. Công cụ và công nghệ sử dụng

Bảng 1 liệt kê stack của module (từ requirements.txt). Dưới đây là các lựa chọn then chốt, nêu việc cần làm trước, công cụ sau:

- **Truy xuất hybrid.** Vector kNN/HNSW và BM25 chung một store nên ghép điểm dễ, khỏi vector DB riêng: **OpenSearch** [5] (opensearch-py).

- **Agent và sinh câu trả lời.** Chuẩn hóa agent, tool, streaming; lo OCR, embedding, tóm tắt, chat: **Google ADK** [7] (google-adk) và **Gemini** [8] (google-genai).
- **Xử lý nền.** Đẩy ingest nặng ra chạy nền, không chặn request: **Celery + Redis** [9].
- **Chuẩn bị tài liệu.** Chunk văn bản, trích text kèm số trang, tải file gốc, chuyển Word sang PDF: **langchain-text-splitters** (RecursiveCharacterTextSplitter), **pypdf**, **boto3** (S3), **LibreOffice**.

Bảng 1: Tech-stack của project chatbot (từ requirements.txt).

Thành phần	Phiên bản	Dùng cho
Django + DRF	5.2 / 3.15+	Web framework, REST API (JSON-only)
Celery + Redis	5.4 / 5.0	Hàng đợi ingest & hậu lượt chat
google-genai (Gemini)	1.x	OCR, embedding, summary, sinh câu trả lời
google-adk	1.x	Agent + tool RAG + Runner streaming SSE
opensearch-py	2.4+	Index/truy xuất hybrid (kNN/HNSW + BM25)
langchain-text-splitters	0.2+	Chunk văn bản (RecursiveCharacterTextSplitter)
pypdf	4.x	Trích text PDF theo trang
boto3	1.34+	Tải file gốc từ S3/MinIO
litellm	1.50+	Backend Ollama cho ADK
lightrag-hku, neo4j, asyncpg	–	Knowledge Graph (cấu hình bật, fail-safe)

VIII. Thực nghiệm và đánh giá mô hình

Nhóm chia đánh giá thành **3 tầng**: truy xuất, sinh câu trả lời và hệ thống, để biết lỗi ở đâu — hệ *lấy nhầm tài liệu* hay *lấy đúng mà vẫn trả lời sai*. Mỗi câu hỏi có sẵn đoạn nguồn *đúng* (golden), so với *top-5* đoạn hệ trả về; tính đúng khi trùng tài liệu (và trùng trang nếu golden ghi trang). Mỗi chỉ số tính theo câu rồi lấy trung bình. Số liệu ở Bảng 2 và Bảng 3.

Tầng truy xuất — hệ có lấy đúng đoạn *golden* không:

- **Hit@5** — top-5 có *ít nhất một* đoạn đúng không (Có/Không).
- **Recall@5** — lấy được bao nhiêu phần các đoạn đáng ra phải lấy; phạt *bỏ sót*.
- **Precision@5** — trong 5 đoạn, bao nhiêu thật sự hữu ích; phạt *lấy thừa*.

Tầng sinh câu trả lời — một mô hình khác chấm câu trả lời so với đáp án mẫu (LLM-as-judge, dùng gemini-2.5-flash):

- **Faithfulness** (bám nguồn) — có *bám đúng tài liệu lấy về* không; chống *bịa*.

- **Answer Relevance** — có trả lời trùng câu hỏi không.
- **Correctness** — khớp nghĩa với đáp án mẫu đến mức nào.

Tầng hệ thống — hệ chạy nhanh và ổn định đến đâu:

- **TTFT** (time-to-first-token) — đến khi *mẫu chữ đầu tiên* hiện ra; quyết định cảm nhận nhanh/chậm vì hệ hiện dần từng phần (SSE).
- **Latency** — tổng thời gian đến khi trả lời *xong* (p50 điển hình, p95 lúc tệ).
- **Tỉ lệ thành công** — bao nhiêu câu trả lời được mà không lỗi.
- **Thời gian ingest** — xử lý xong một tài liệu (tải, trích chữ hoặc OCR, cắt đoạn, tạo vector, ghi index); đo trên *mẫu vài tài liệu*.

Cách tạo bộ câu hỏi kiểm thử. Rút câu từ tài liệu thật, ghi golden theo (document_id, trang) kèm đáp án mẫu, rồi chạy lần lượt từng cấu hình trên cùng bộ câu và đo.

Thiết lập thực nghiệm đã chạy

Golden QA tự soạn: **28 câu hỏi** (đa ngôn ngữ) từ **9 tài liệu** khóa học thật (782 chunk), mỗi câu có (document_id, trang) và đáp án mẫu. Tầng truy xuất chấm tự động (Recall@5/Precision@5/Hit@5) qua eval_rag; tầng sinh cho chạy *agent ADK thật* rồi chấm Faithfulness/Answer-Relevance/Correctness bằng **LLM-as-judge** (gemini-2.5-flash) và đo thời gian. Mỗi cấu hình chạy 1 lần; lúc đo tắt truy hồi KG để chỉ đo đường hybrid.

Bảng 2: **Tầng truy xuất** trên 28 câu golden QA (9 tài liệu, 782 chunk), 3 cấu hình. “Triển khai” = search() thực tế: hybrid BM25+kNN, RRF, sà cosine 0.80; rerank cross-encoder cấm sẵn nhưng *không* chạy vì endpoint trống.

Cấu hình	Recall@5	Precision@5	Hit@5
Baseline (BM25 thuần)	0.411	0.093	0.464
Hybrid (BM25+kNN+RRF)	0.821	0.200	0.857
Triển khai (hybrid+RRF+sà cosine)	0.875	0.214	0.893

Bảng 3: **Tầng sinh + hệ thống**, cấu hình triển khai (28 câu). Faithfulness/Answer-Relevance/Correctness chấm bằng **LLM-as-judge** (gemini-2.5-flash).

Nhóm / Chỉ số	Giá trị
<i>Sinh — LLM-as-judge (thang 0–1)</i>	
Faithfulness (bám nguồn, chống bịa)	1.000
Answer Relevance (trúng câu hỏi)	0.993
Correctness (so đáp án mẫu)	0.929
<i>Hệ thống</i>	
TTFT (p50 / p95)	5.15 / 5.82 s
Latency (p50 / p95)	5.36 / 5.97 s
Tỉ lệ thành công	28/28 (100%)
Thời gian ingest (mẫu)	7.5 s (text) – 255 s (PDF cần OCR)

Nhận xét. (1) *Hybrid* vượt xa BM25 thuần (Recall@5 0.411 → 0.821; Hit@5 0.464 → 0.857), nhờ tìm theo nghĩa (kNN) + RRF, nhất là câu paraphrase không trùng từ khóa. (2) Triển khai nhỉnh hơn hybrid thô (Recall +0.054, Hit +0.036), do sần cosine và chọn top-*k* chứ *không* do rerank cross-encoder (chưa chạy). (3) **Hit@5** = 0.893 (≈89% câu có đoạn đúng trong top-5). (4) Tầng sinh tốt: **Faithfulness** 1.0, **Answer-Relevance** 0.993, **Correctness** 0.929. Hệ mạnh ở *tra cứu sự kiện, định nghĩa, yếu ở suy luận nhiều bước và câu hỏi mơ hồ* — đúng nhóm câu mà **LightRAG/KG** (search_knowledge_graph, mode mix) muốn hỗ trợ.

Lưu ý trung thực. Bộ kiểm thử nhỏ ($n = 28$), golden tự soạn từ tài liệu thật, nên số liệu *minh họa có cơ sở*, chưa phải bằng chứng thống kê mạnh. **Precision@5** (0.214) bị *trần metric*: đa số câu chỉ có một trang golden nên tối đa ≈ 0.2, thấp không có nghĩa truy hồi kém. **Faithfulness, Answer-Relevance, Correctness** chấm bằng **LLM-as-judge** nên chỉ đọc như *chỉ báo*; riêng Faithfulness 1.0 hiểu là “rất ít bịa trên bộ này”. **Thời gian ingest** đo trên *mẫu 2 tài liệu*: dạng chữ ≈7.5 s, PDF slide nhiều ảnh ≈255 s (phần lớn do **OCR Gemini chạy từng trang**).

VIII.1. Đánh giá LightRAG/KG riêng (A/B trên câu hỏi quan hệ)

Với 28 câu đơn bước ở trên, agent tự chọn hầu như không dùng đường đồ thị nên KG không thể hiện gì trong Bảng 2. Lợi ích thật của KG nằm ở câu hỏi *quan hệ*, cần nối nhiều bước. Nhóm soạn riêng **8 câu hỏi quan hệ**, mỗi câu nối khái niệm giữa các tài liệu (vai trò ↔ sự kiện trong Scrum, normalization ↔ CRUD), rồi chạy A/B trên cùng một agent: một lần *bật*, một lần *tắt* KG.

Phát hiện và sửa một lỗi runtime. Ban đầu KG trả về rỗng dù đã *bật*: tool truy vấn KG gọi `asyncio.run()` trong khi runner của agent đang giữ một event loop chạy dở, gây lỗi “*cannot be called from a running event loop*” nhưng bị fail-safe nuốt mất nên khó thấy. Nhóm cho truy vấn KG chạy trong một *luồng riêng*; sau khi sửa, KG trả về khoảng 97K ký tự ngữ cảnh thực

thể và quan hệ, và đường đồ thị mới thật sự hoạt động.

Bảng 4: A/B **LightRAG/KG** trên 8 câu hỏi quan hệ; cùng agent, chỉ khác bật/tắt đường đồ thị, chấm bằng LLM-as-judge.

Chỉ số (8 câu quan hệ)	KG tắt	KG bật
Correctness (so đáp án mẫu)	0.691	0.938
Answer Relevance	0.950	1.000
Faithfulness (bám nguồn)	0.841	0.694

Nhận xét A/B. Bật KG giúp **Correctness** tăng rõ ($0.691 \rightarrow 0.938$) và Answer-Relevance tăng ($0.950 \rightarrow 1.0$) nhờ ngữ cảnh thực thể và quan hệ bổ sung. Ngược lại, **Faithfulness** giảm ($0.841 \rightarrow 0.694$) vì kết quả KG *không kèm citation*, đúng với hạn chế đã nêu. **Lưu ý trung thực:** mẫu rất nhỏ ($n = 8$) và agent chỉ *tự gọi* KG ở 2/8 câu, cộng thêm tính ngẫu nhiên của LLM, nên kết quả chỉ là *gợi ý*. Hướng tiếp theo: tinh chỉnh để agent chủ động dùng KG cho câu quan hệ và gán citation cho kết quả KG.

IX. Phân tích lỗi và hạn chế

- **Truy xuất sai hoặc thiếu:** đoạn cắt đứt ý, embedding chưa hợp tiếng Việt, hoặc câu hỏi mơ hồ. *Khắc phục:* cắt đoạn khéo hơn, thử embedding tiếng Việt, viết lại câu hỏi trước khi tìm.
- **OCR yếu:** trang scan mờ, bảng và công thức dễ ra chữ rác. *Khắc phục:* chỉnh ngưỡng nhận dạng và kiểm lại chất lượng chữ.
- **Trả lời bịa hoặc lệch nguồn:** mô hình bịa khi ngữ cảnh không đủ. *Khắc phục:* buộc “chỉ trả lời theo nguồn”, lọc đoạn kém liên quan bằng ngưỡng điểm, trả “không tìm thấy” khi thiếu căn cứ.
- **Hội thoại:** lịch sử và bộ nhớ dài hạn đôi khi nạp sai ngữ cảnh, mất mạch khi hỏi nối tiếp. *Khắc phục:* siết ngưỡng lấy ký ức và giảm số lượt lịch sử đưa vào.
- **Hệ thống:** tài liệu lớn thì độ trễ cao, chi phí gọi mô hình tăng, số tài liệu giữa các chủ đề chưa cân.

Hạn chế hiện tại. Bốn điểm còn dang dở.

(1) **Viết lại câu hỏi** mới có cấu hình, *chưa* hiện thực trong mã.

(2) **Rerank** (xếp lại thứ hạng đoạn) cần dịch vụ cross-encoder cắm ngoài; chưa có thì hệ thống bỏ qua bước này.

(3) **Đường đồ thị tri thức (LightRAG/KG)** đã bật trong cấu hình và chạy an toàn: lỗi khi dựng hay truy vấn chỉ ghi log chứ không làm hỏng tài liệu, và khi đồ thị trả rỗng thì hệ thống vẫn

trả lời qua đường tìm kiếm thường. Đánh đổi khi bật: phải dựng thêm **PostgreSQL và Neo4j**, tốn chi phí gọi mô hình để trích thực thể lúc nạp, và kết quả đồ thị không kèm trích dẫn nên không sinh được citation — đây là lý do độ bám nguồn (Faithfulness) giảm khi dùng đồ thị (mục VIII.1.); chất lượng cũng phụ thuộc bước trích thực thể. *Trong thực nghiệm này*, nhóm đã dựng đồ thị và nạp đủ 9 tài liệu (**4407 thực thể / 6258 quan hệ**), **phát hiện và sửa** một lỗi khiến đồ thị trả rỗng trong luồng chat, rồi chạy A/B trên 8 câu quan hệ (bật đồ thị giúp Correctness tăng 0.691 → 0.938, mục VIII.1.). Dù vậy tác nhân chỉ tự tìm đến đồ thị ở số ít câu (2/8), nên tính chính việc chọn công cụ là hướng tiếp theo.

(4) Vài tham số truy xuất còn ghi cứng trong mã, chưa chỉnh nóng được qua tệp cấu hình.

X. Kết luận và hướng phát triển

Kết luận. Nhóm đã dựng một trợ lý hỏi–đáp tài liệu khóa học **chạy được** đầu-cuối. Khâu nạp đọc chữ (kể cả OCR trang scan), cắt đoạn kèm câu tóm tắt bối cảnh, chuyển mỗi đoạn thành vector 768 chiều rồi đưa vào kho tìm kiếm (OpenSearch, chỉ mục HNSW). Khâu truy xuất kết hợp tìm theo từ khóa và theo ý nghĩa, hợp nhất bằng RRF ($k=60$) rồi lọc đoạn kém liên quan; bước xếp lại thứ hạng bằng cross-encoder đã cắm sẵn nhưng chưa bật vì thiếu dịch vụ ngoài. Câu trả lời được viết bằng Gemini (qua Google ADK), trả về hiện dần từng phần.

Nhóm cũng hiện thực thêm **đường truy hồi thứ hai bằng đồ thị tri thức (LightRAG/KG)** [1]: rút thực thể và quan hệ, lưu trên PostgreSQL/Neo4j, bổ trợ đường tìm kiếm chính khi đủ hạ tầng. Đường này đã bật và chạy an toàn, nhưng trên bộ kiểm thử một bước thì tác nhân chưa gọi tới (0/28). Toàn bộ tổ chức nhiều lớp, chạy nền bằng Celery, mọi tham số then chốt đều là *số thật* lấy từ mã nguồn.

Hướng phát triển. Định hướng chính là giảm phụ thuộc dịch vụ đám mây (Gemini, Google ADK), chuyển dần sang **tự vận hành bằng mã nguồn mở** để chủ động hơn về chi phí, dữ liệu và tính chính — bám lộ trình kiến thức khóa học. Việc chuyển đổi chia thành bốn nhánh:

- **Đọc tài liệu và OCR:** thay dần khâu nhận dạng của Gemini bằng bộ OCR tự vận hành cho PDF scan và ảnh, chuẩn hóa và làm sạch chữ (tách đoạn, xử lý bảng, gắn nhãn thực thể), đối chiếu chất lượng trước khi thay hẳn (công cụ: Tesseract, hoặc YOLOv10 kèm CNN/Transformer).
- **Tạo vector (embedding):** chuyển sang mô hình chạy nội bộ 768 chiều. Phần chọn nguồn **đã có sẵn trong mã** nhưng chưa bật mặc định; tiếp theo là đo đối chứng với Gemini và tối ưu tốc độ (gộp lô, dùng GPU, nén mô hình) (công cụ: Ollama, nomic-embed-text).
- **Mô hình viết câu trả lời:** chuyển sang mô hình mở chạy nội bộ. Phần chọn nguồn **đã tích hợp** nhưng chưa bật mặc định; tiếp theo là phục vụ hiệu quả hơn, nén mô hình, đánh giá độ chính xác và độ an toàn trước khi thay hẳn (công cụ: Ollama với qwen2.5, phục vụ bằng vLLM, nén bằng quantization hoặc ONNX/TensorRT).
- **Điều phối tác nhân:** thay Google ADK bằng luồng tự viết cho vòng *tìm đoạn* → *ghép prompt* → *sinh câu trả lời*, mở rộng sang nhiều tác nhân khi cần, kèm ghi log và đánh giá liên tục (công cụ: Python thuần với Tool Calling và ReAct, Multi-agent, MCP).

Song song, nhóm **cải thiện phân lỗi** của trợ lý, độc lập với việc tự vận hành: bắt **viết lại câu hỏi** (đã có cấu hình) để tìm nhiều đoạn liên quan hơn; dựng dịch vụ **xếp lại thứ hạng đoạn** và dùng embedding tốt hơn cho **tiếng Việt**; **đánh giá định lượng đường đồ thị tri thức** cho câu hỏi nhiều bước và chỉnh việc chọn công cụ để tác nhân chủ động dùng đồ thị [1], [6]; trích **trích dẫn** rõ trong câu trả lời, lưu tạm câu hỏi phổ biến, đánh giá tự động liên tục, rút ngắn thời gian ra chữ đầu tiên (công cụ rerank: bge-reranker-v2-m3).

XI. Tài liệu tham khảo

- [1] HKUDS. “LightRAG: Simple and fast retrieval-augmented generation (github)”, Accessed: Jun. 28, 2026. [Online]. Available: <https://github.com/HKUDS/LightRAG>
- [2] Neo4j. “Neo4j graph database documentation”, Accessed: Jun. 28, 2026. [Online]. Available: <https://neo4j.com/docs/>
- [3] pgvector. “Pgvector: Open-source vector similarity search for PostgreSQL”, Accessed: Jun. 28, 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [4] LangChain. “Recursively split by character (RecursiveCharacterTextSplitter)”, Accessed: Jun. 28, 2026. [Online]. Available: https://python.langchain.com/docs/how_to/recursive_text_splitter/
- [5] OpenSearch Project. “Opensearch: Vector search and k-NN”, Accessed: Jun. 27, 2026. [Online]. Available: <https://opensearch.org/docs/latest/search-plugins/knn/index/>
- [6] Microsoft. “GraphRAG documentation”, Accessed: Jun. 28, 2026. [Online]. Available: <https://microsoft.github.io/graphrag/>
- [7] Google. “Agent development kit (ADK) documentation”, Accessed: Jun. 27, 2026. [Online]. Available: <https://google.github.io/adk-docs/>
- [8] Google. “Gemini API documentation”, Accessed: Jun. 28, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [9] Celery Project. “Celery: Distributed task queue”, Accessed: Jun. 27, 2026. [Online]. Available: <https://docs.celeryq.dev/>

Phụ lục

1. **AIVIETNAM:** Thông tin về AIVIETNAM có thể đọc thêm tại [đây](#).
2. **GitHub Repository:** github.com/AIVIETNAM-AIO-NG-LINH/aio-2026.
3. **Demo App / Website:** aio-it.dev.
4. **Demo Video:** [RAG Chatbot — Overfitting Club \(Demo\)](#).
5. **Dataset / Tài liệu khóa học:** 9 tài liệu khóa học AIO 2026 (Python cơ bản: vòng lặp & rẽ nhánh/chatbot luật; SQL & ERD; Coding Methodology; Image & Video Data; Teamwork Tools; hướng dẫn làm video; chính sách bảo mật) — nạp thành 782 chunk trên rag-index.